Ford Joel

In this implementation, I used std::variant to represent either an empty tree or a node containing a value and two subtrees. I used std::visit and lambda expressions to check which type was stored, which is similar to pattern matching in OCaml. I also added inorder, preorder, and postorder traversals. Compared to the BST we used in CS 372, this version uses std::variant instead of relying on regular pointers and checking for nullptr. Both implementations work similarly, but std::variant makes the different possible states of the tree more explicit. The regular BST implementation is simpler to understand, while this version is more similar to how OCaml handles trees.